# Laboratory 1:

Link to the BigQuery queries (all eleven prompt):

https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1spurdue-ms-ds-bruno-cavagnaro!2sus-central1!3sec55d4a6-9c9a-44ab-b252-6d7c3c943a7e!2e1




LIST OF QUERIES:

```sql
--- First query - data cleaning of 0

SELECT
  Date,
  Open,
  High,
  Low,
  Close,
  Volume,
  COALESCE(Dividends, 0) AS Dividends,
  COALESCE(`Stock Splits`, 0) AS `Stock Splits`
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock`
LIMIT 1000;
```

```sql
--- Second query - data cleaning of 0

SELECT
  CAST(Date AS DATE) AS Date,
  Open,
  High,
  Low,
  Close,
  Volume,
  COALESCE(Dividends, 0) AS Dividends,
  COALESCE(`Stock Splits`, 0) AS `Stock Splits`
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock`
LIMIT 1000;

-- Third query - Identify outliers

SELECT
  t1.*,
  (t1.Close - t2.avg_close) / t2.stddev_close AS z_score_close,
  (t1.Volume - t2.avg_volume) / t2.stddev_volume AS z_score_volume
FROM
  `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock` AS t1,
  (
    SELECT
      AVG(Close) AS avg_close,
      STDDEV(Close) AS stddev_close,
      AVG(Volume) AS avg_volume,
      STDDEV(Volume) AS stddev_volume
    FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock`
  ) AS t2
WHERE
  ABS((t1.Close - t2.avg_close) / t2.stddev_close) > 3
  OR ABS((t1.Volume - t2.avg_volume) / t2.stddev_volume) > 3
ORDER BY Date;

-- Fourth query - Check for duplicate records

SELECT
  t.`Date`,
  t.Open,
  t.High,
  t.Low,
  t.Close,
  t.Volume,
  t.Dividends,
  t.`Stock Splits`,
  COUNT(*) AS record_count
```

```sql
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock` AS t
GROUP BY
  t.`Date`, t.Open, t.High, t.Low, t.Close, t.Volume, t.Dividends,
  t.`Stock Splits`
HAVING COUNT(*) > 1
ORDER BY t.`Date`

-- Fifth query - Average values for all numerical columns

SELECT
  AVG(t.Open) AS avg_open,
  AVG(t.High) AS avg_high,
  AVG(t.Low) AS avg_low,
  AVG(t.Close) AS avg_close,
  AVG(t.Volume) AS avg_volume,
  AVG(t.Dividends) AS avg_dividends,
  AVG(t.`Stock Splits`) AS avg_stock_splits
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock` AS t;


-- Sixth Query - Average values of numeric columns for a specific year like 2023

SELECT
  AVG(t.Open) AS average_open,
  AVG(t.High) AS average_high,
  AVG(t.Low) AS average_low,
  AVG(t.Close) AS average_close,
  AVG(t.Volume) AS average_volume,
  AVG(t.Dividends) AS average_dividends,
  AVG(t.`Stock Splits`) AS average_stock_splits
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock` AS t
WHERE EXTRACT(YEAR FROM t.Date) = 2023;


-- Seventh Query - Find the maximum and minimum by month


SELECT
  DATE_TRUNC(t.Date, YEAR) AS year,
  DATE_TRUNC(t.Date, MONTH) AS month,
  MAX(t.Open) AS max_open,
  MIN(t.Open) AS min_open,
  MAX(t.High) AS max_high,
  MIN(t.High) AS min_high,
  MAX(t.Low) AS max_low,
  MIN(t.Low) AS min_low,
  MAX(t.Close) AS max_close,
  MIN(t.Close) AS min_close,
```

```sql
  MAX(t.Volume) AS max_volume,
  MIN(t.Volume) AS min_volume,
  MAX(t.Dividends) AS max_dividends,
  MIN(t.Dividends) AS min_dividends,
  MAX(t.`Stock Splits`) AS max_stock_splits,
  MIN(t.`Stock Splits`) AS min_stock_splits
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock` AS t
GROUP BY DATE_TRUNC(t.Date, YEAR), DATE_TRUNC(t.Date, MONTH)
ORDER BY DATE_TRUNC(t.Date, YEAR), DATE_TRUNC(t.Date, MONTH);


-- Eith query - Counting the number of trading days

SELECT COUNT(DISTINCT Date) AS number_of_trading_days
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock`;


-- Nine query - Calculate high trading volume days

SELECT t1.date, t1.volume
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock` AS t1
WHERE
  t1.volume >= (
    SELECT PERCENTILE_CONT(t2.volume, 0.75) OVER ()
    FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock` AS t2
    LIMIT 1
  )
ORDER BY t1.date;


-- Ten query - calculate daily percentage count

SELECT
  Date,
  Close,
  LAG(Close, 1) OVER (ORDER BY Date) AS previous_day_close,
  (
    (Close - LAG(Close, 1) OVER (ORDER BY Date))
    / LAG(Close, 1) OVER (ORDER BY Date))
    * 100 AS daily_percentage_change
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock`
ORDER BY Date;

-- Eleve query - calculate the stock performance over time

SELECT
  Date,
  Close,
```

```
  AVG(Close)
    OVER (ORDER BY Date ASC ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
    AS seven_day_moving_average
FROM `purdue-ms-ds-bruno-cavagnaro.lab1_eda.nvda_stock`
ORDER BY Date;
```